



IF3170 Inteligensi Artifisial

Institut Teknologi Bandung

Constraints Satisfaction Problem (CSP)

Semester I TA 2026/2027





Outline

- Review:
 - What is AI (4 Approaches), PEAS, Property of Task Environment, Agent Level
 - Local Search: Hill Climbing, Simulated Annealing, Genetic Algorithm
 - Adversarial Search
- Constraints Satisfaction Problem (CSP)
 - What
 - Constraint Graph
 - Inference in CSP
 - Backtracking Search for CSP
 - Interleaving Search and Inference in CSP





Constraint satisfaction problems (CSPs)

- Standard search problem:
 - **state** is a "black box" – any data structure that supports successor function, heuristic function, and goal test
- CSP:
 - **state** is defined by **variables** X_i with **values** from **domain** D_i
 - **goal test** is a set of **constraints** specifying allowable combinations of values for subsets of variables
- Simple example of a **formal representation language**
- Allows useful **general-purpose** algorithms with more power than standard search algorithms



Example: Map-Coloring



- **Variables** WA, NT, Q, NSW, V, SA, T
- **Domains** $D_i = \{\text{red}, \text{green}, \text{blue}\}$
- **Constraints**: adjacent regions must have different colors
- e.g., $WA \neq NT$, or (WA, NT) in $\{(\text{red}, \text{green}), (\text{red}, \text{blue}), (\text{green}, \text{red}), (\text{green}, \text{blue}), (\text{blue}, \text{red}), (\text{blue}, \text{green})\}$



Example: Map-Coloring

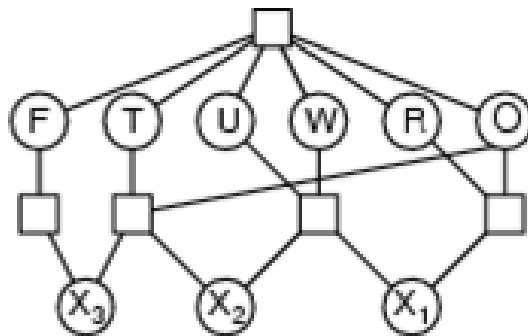
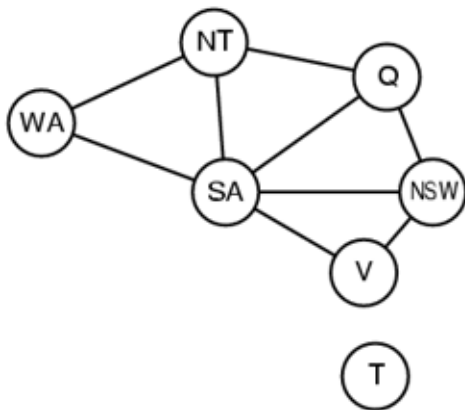


- **Solutions** are **complete** and **consistent** assignments, e.g., WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green
- 6 variabel, 3 colours : 3^6 complete assignments



Constraint graph

- **Binary CSP:** each constraint relates two variables
- **Constraint graph:** nodes are variables, arcs are constraints
- **Constraint hypergraph:** nodes are variables, squares (hypernodes) represent n-ary constraint





Varieties of CSPs

- Discrete variables
 - finite domains:
 - n variables, domain size d : $O(d^n)$ complete assignments
 - e.g., Boolean CSPs, incl. Boolean satisfiability
 - infinite domains:
 - integers, strings, etc.
 - e.g., job scheduling, variables are start/end days for each job
 - need a constraint language, e.g., $StartJob_1 + 5 \leq StartJob_3$
 - Suppose there is requirement to get the whole assembly done in m minutes. We can achieve that by limiting the domain: $D_i = \{1, 2, 3, \dots, m\}$: finite
- Continuous variables
 - e.g., start/end times for Hubble Space Telescope observations
 - linear constraints solvable in polynomial time by linear programming

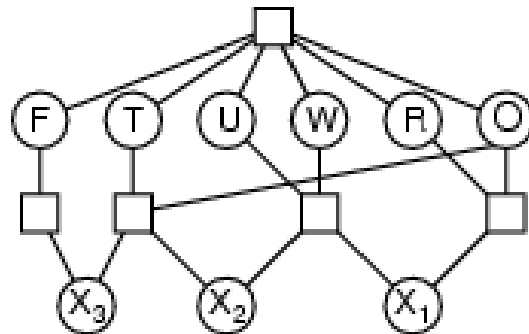


Varieties of constraints

- **Unary** constraints involve a single variable,
 - e.g., $SA \neq \text{green}$
- **Binary** constraints involve pairs of variables,
 - e.g., $SA \neq WA$
- **Higher-order** constraints involve 3 or more variables,
 - e.g., cryptarithmic column constraints
- **Preference**
 - Certain tolerance embedded in constraints, each option has cost $\rightarrow$ optimal solution
 - Constraint optimization problem (COP) : linear programming



$$\begin{array}{r} \text{TWO} \\ + \text{TWO} \\ \hline \text{FOUR} \end{array}$$



- **Variables:** $F, T, U, W, R, O, X_1, X_2, X_3$
- **Domains:** $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- **Constraints:** $AllDiff(F, T, U, W, R, O)$
 - $O + O = R + 10 \cdot X_1$
 - $X_1 + W + W = U + 10 \cdot X_2$
 - $X_2 + T + T = O + 10 \cdot X_3$
 - $X_3 = F, T \neq 0, F \neq 0$



Real-world CSPs

- Assignment problems
 - e.g., who teaches what class
- Timetabling problems
 - e.g., which class is offered when and where?
- Transportation scheduling
- Factory scheduling
- Automobile Product Configuration
- Notice that many real-world problems involve real-valued variables



Inference in CSP: Constraint Propagation

- In CSP : algorithm can search or do specific inference (constraint propagation)
- Constraint Propagation: reduce value of a variable's domain
-> can reduce value for another variable
- Constraint Propagation may be intertwined with search, or it may be done as a preprocessing step; Key: Local consistency
 - Node Consistency
 - Arch Consistency
 - Path Consistency
 - K- Consistency

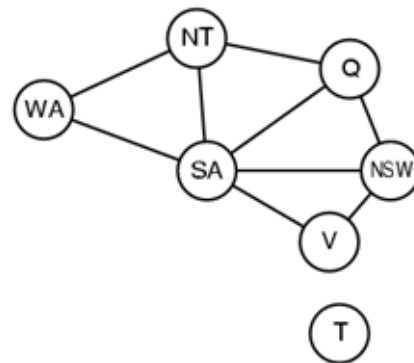


Node Consistency

All the values in the variable's domain satisfy the variable's **unary constraints**

Example: $SA \neq \text{green}$

$SA = \{\text{red, blue}\}$

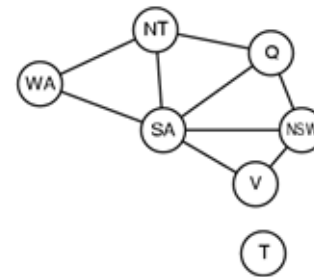


A network is node-consistent if every variable in the network is node-consistent



Arc Consistency (AC)

A variable in a CSP is **arc-consistent** if every value in its domain satisfies the variable's binary constraints



Example: $SA \neq WA$

$(SA, WA) = \{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\}$

Has no effect in this example (no reduction in the domain)

A network is arc-consistent if every variable is arc consistent with every other variable

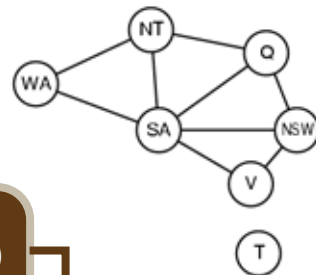


Path Consistency (PC)

Arc Consistency: solve the problem if each variable has only 1 value left after the process OR finds that CSP can not be solved

Does not work for map coloring with only 2 values in the domain

(Path Consistency)



$\{X_i, X_j\}$ is path-consistent to X_m if:

- Assignment $\{X_i = a, X_j = b\}$ consistent with constraints on $\{X_i, X_j\}$
- There is assignment to X_m that satisfies constraints on $\{X_i, X_m\}$ and $\{X_m, X_j\}$.

Example: Coloring Map with 2 colors (red, blue)

PC: $\{WA, SA\}$ wrt NT

$\{WA = \text{red}, SA = \text{blue}\}$ or
 $\{WA = \text{blue}, SA = \text{red}\}$

No valid choice for NT

Eliminate both assignment? No solution



K-Consistency

A CSP is k -consistent if: any set of $k - 1$ variables & any consistent assignment to those variables, there is a consistent value to be assigned to k^{th} variable

1-consistency: given empty set, can make any set of one variable consistent

2-consistency = Arc Consistency

3-consistency = Path Consistency



Backtracking Search for CSP

- Many CSPs cannot be solved by inference alone; there comes a time when we must search for a solution
 - Partial assignment: Backtracking Search
 - Complete assignment: Local Search
- Let's start with the **Backtracking straightforward approach**, then fix it
- Every solution appears at depth n with n variables
-> use depth-first search
- Path is irrelevant
- Variable assignments are **commutative**, i.e.,
 - [WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each node
- Depth-first search for CSPs with single-variable assignments is called backtracking search
Backtracking search is the basic uninformed algorithm for CSPs



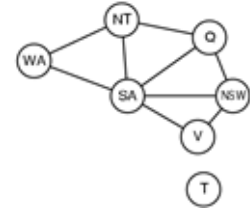
Backtracking search

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment then
      add { var = value } to assignment
      inferences ← INFERENCE(csp, var, value)
      if inferences ≠ failure then
        add inferences to assignment
        result ← BACKTRACK(assignment, csp)
        if result ≠ failure then
          return result
      remove { var = value } and inferences from assignment
  return failure
```

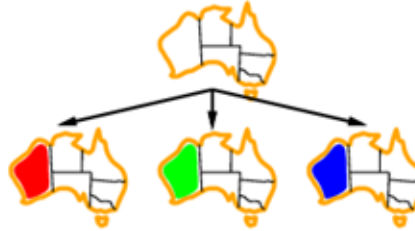
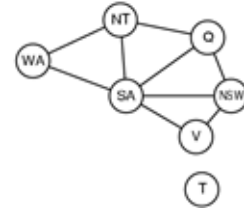


Backtracking example



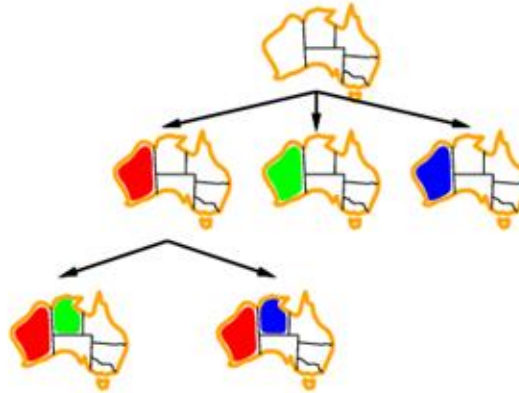
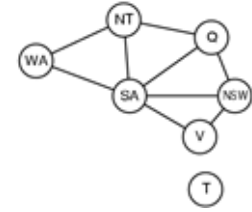


Backtracking example



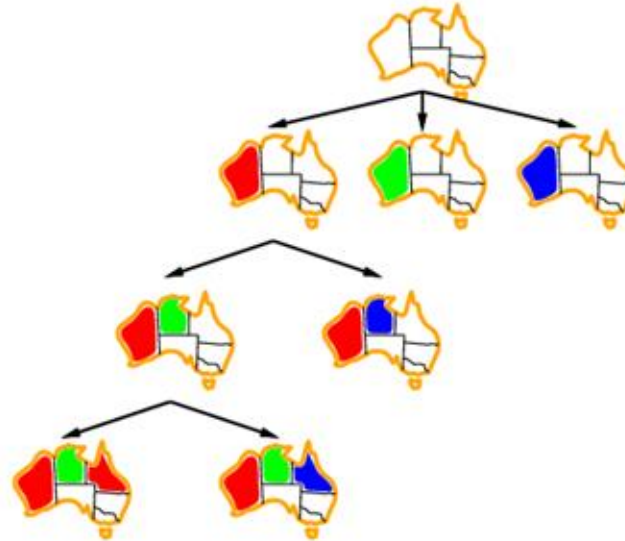
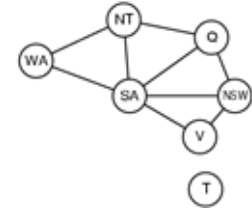


Backtracking example





Backtracking example





Improving backtracking efficiency

- **General-purpose** methods can give huge gains in speed:
 - Which variable should be assigned next?
 - Random , static order ?
 - In what order should its values be tried?
 - Can we detect inevitable failure early?



Variable Ordering

1. Static order: $\{WA, NT, SA, Q, NSW, V, T\}$



2. Most constrained variable:

- choose the variable with the fewest legal values
a.k.a. **minimum remaining values (MRV)** heuristic or fail-first heuristic
- If some variable X has no legal values left, the MRV heuristic will select X and failure will be detected immediately

After WA=red and
NT=green,
SA or Q?

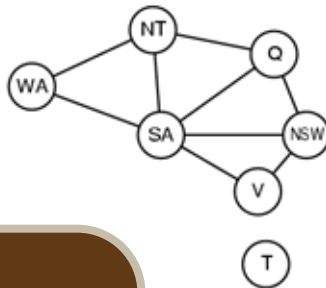
SA={blue}
Q={blue, red}

Choose SA (has
minimum
remaining value)



First Variable: Most constrained variable

- Tie-breaker among most constrained variables
- Most constraining variable:
 - choose the variable with the most constraints on remaining variables (degree heuristic)



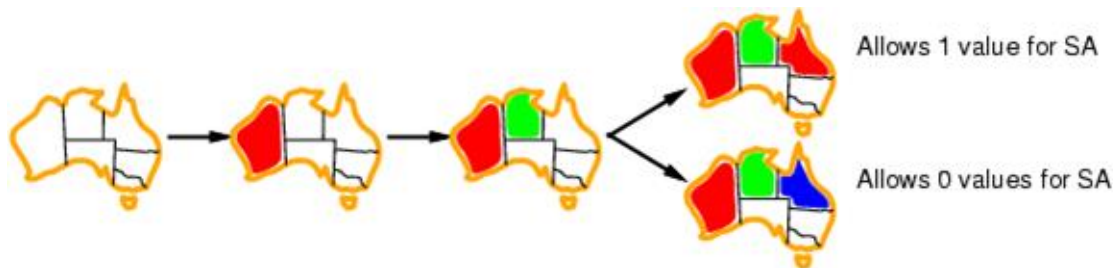
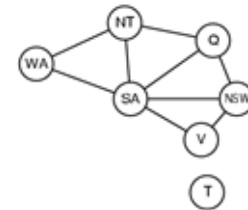
SA: 5
NT, Q, NSW: 3
WA, V: 2
T: 0

Choose SA First!!



Value Ordering: Least constraining value

- Given a variable, choose the least constraining value:
 - the one that rules out the fewest values in the remaining variables
- Combining these heuristics makes 1000 queens feasible



Value Ordering is irrelevant if we want to have all possible solutions



Interleaving Search and Inference in CSP

- Inference can be done before searching
- Interleaving search and inference -> detect failure early
 - Forward Checking: establishes arc consistency for binary constraint
 - Constraint Propagation: Maintaining Arc Consistency (MAC)

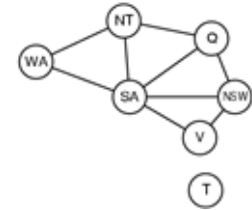


26



Forward checking

- **Idea:**
 - Keep track of remaining legal values for unassigned variables
 - Terminate search when any variable has no legal values





Forward checking

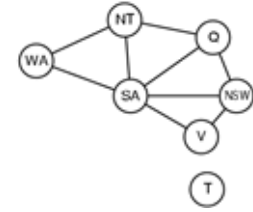
- **Idea:**
 - Keep track of remaining legal values for unassigned variables
 - Terminate search when any variable has no legal values



WA	NT	Q	NSW	V	SA	T



Forward checking



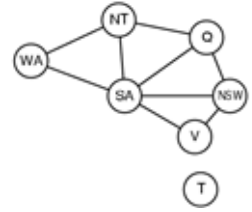
- **Idea:**
 - Keep track of remaining legal values for unassigned variables
 - Terminate search when any variable has no legal values



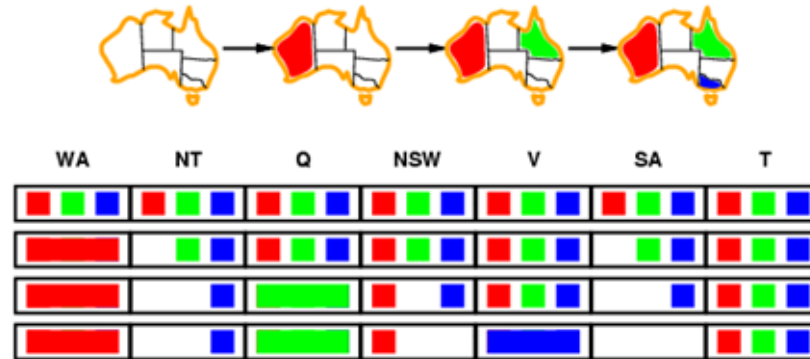
WA	NT	Q	NSW	V	SA	T

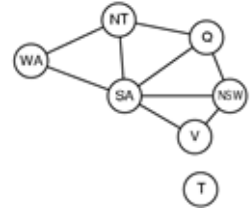


Forward checking

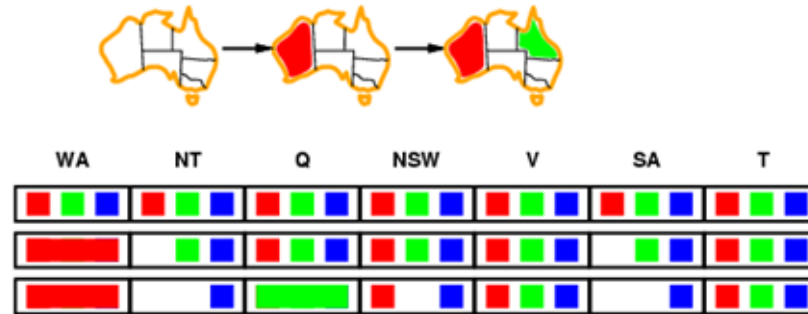


- **Idea:**
 - Keep track of remaining legal values for unassigned variables
 - Terminate search when any variable has no legal values





- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:



- NT and SA cannot both be blue!
Constraint propagation repeatedly enforces constraints locally

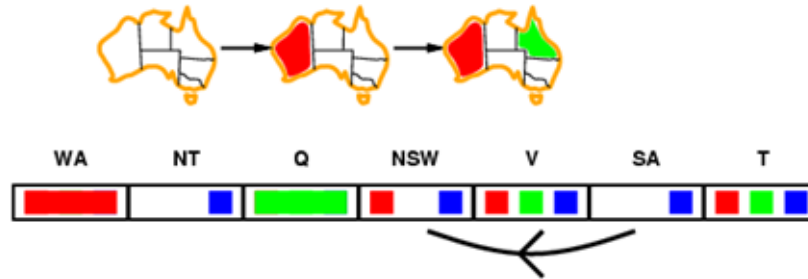
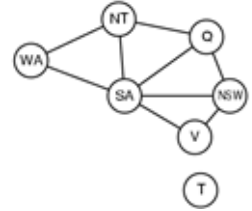


MAC

Simplest form of propagation makes each arc **consistent**

$X \rightarrow Y$ is consistent iff

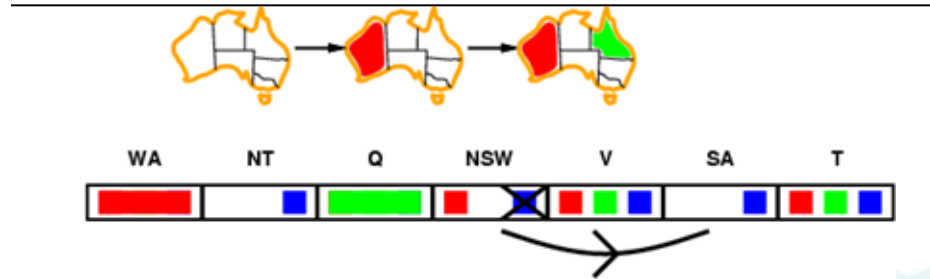
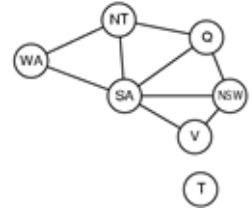
for **every** value x of X there is **some** allowed y





MAC

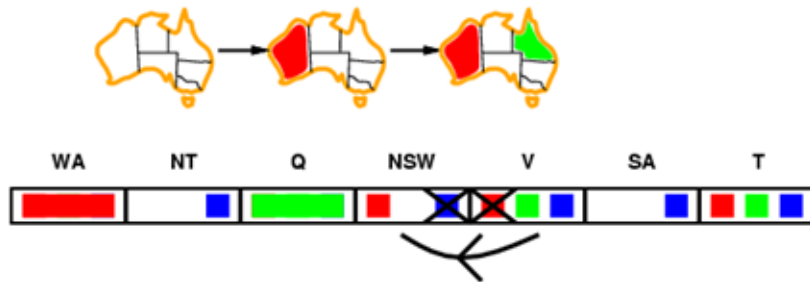
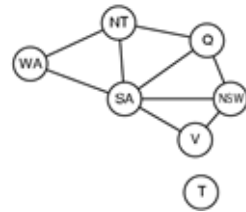
- Simplest form of propagation makes each arc **consistent**
- $X \rightarrow Y$ is consistent iff
 - for **every** value x of X there is **some** allowed y





MAC

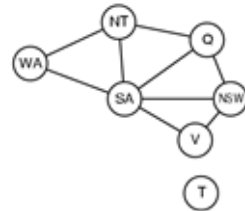
- Simplest form of propagation makes each arc **consistent**
- $X \rightarrow Y$ is consistent iff
 - for **every** value x of X there is **some** allowed y



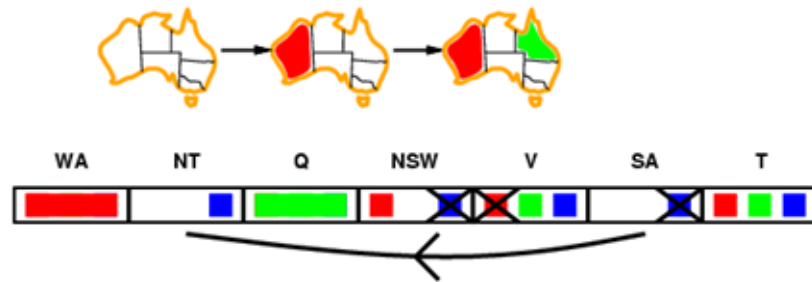
- If X loses a value, neighbors of X need to be rechecked



MAC



- Simplest form of propagation makes each arc **consistent**
- $X \rightarrow Y$ is consistent iff
 - for **every** value x of X there is **some** allowed y



- If X loses a value, neighbors of X need to be rechecked
- Maintain Arc consistency detects failure earlier than forward checking
- Can be run as a preprocessor or after each assignment



Special Constraints

- Handle special constraints
 - Each variable must be assigned to different value
 - Failure detection -> m variable with n values, $m > n$



36



Local search for CSPs

- Hill-climbing, simulated annealing typically work with "complete" states, i.e., all variables assigned
- To apply to CSPs:
 - allow states with unsatisfied constraints
 - operators **reassign** variable values
- Variable selection: randomly select any conflicted variable
- Value selection by **min-conflicts** heuristic:
 - choose value that violates the fewest constraints
 - i.e., hill-climb with $h(n)$ = total number of violated constraints



Min-conflict heuristic

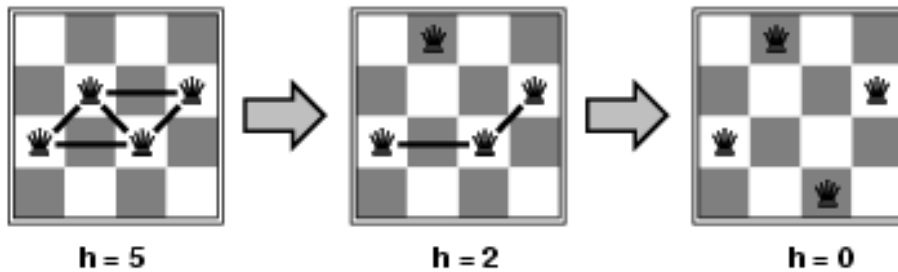
```
function MIN-CONFLICTS(esp, max_steps) returns a solution or failure
  inputs: esp, a constraint satisfaction problem
         max_steps, the number of steps allowed before giving up

  current ← an initial complete assignment for esp
  for i = 1 to max_steps do
    if current is a solution for esp then return current
    var ← a randomly chosen conflicted variable from esp.VARIABLES
    value ← the value v for var that minimizes CONFLICTS(var, v, current, esp)
    set var = value in current
  return failure
```



Example: 4-Queens

- **States:** 4 queens in 4 columns ($4^4 = 256$ states)
- **Actions:** move queen in column
- **Goal test:** no attacks (minimum $h(n)$)
- **Evaluation:** $h(n)$ = number of attacks



- Given random initial state, can solve n -queens in almost constant time for arbitrary n with high probability (e.g., $n = 10,000,000$)



Local Search

- Application:
 - *Online setting* -> scheduling
 - With *backtracking search* consumes more time and more adjustments



40



References

- Stuart J Russell & Peter Norvig, Artificial Intelligence: A Modern Approach, 4th Edition, Prentice-Hall International, Inc, 2022, Textbook
Site: <http://aima.cs.berkeley.edu/> (4th Edition)





Thank You

